Date:

Assignment-7

* Problem Statement
  Implement Stack as an ADT and apply it for different expression conversions (infix to postfix (Any one) prefix to postfix or prefix to infix or postfix to prefix (Any one)

* Objective :
  1. To study stack and its operations
  2. To study the importance of expression conversion.

* Test Condition.

No.	Input	Conversion	Output
1	A+B*C	Infix → Postfix	ABC*+
2	(A+B)*C	Infix → Postfix	AB+C*
3	ABC*+	Postfix → Infix	A+(B*C)
4	AB+C*	Postfix → Infix	(A+B)*C

* Pseudo code

PUSH :
If Stack is full → overflow
Else TOP = TOP + 1
STACK [TOP] = ITEM

POP:

If stack is empty -> underflow
Else ITEM = STACK[TOP]
TOP = TOP - 1

INFIXT TO POSTFIX:
Scan expression left to right
operand -> Add to output
'(' -> Push
')' -> Pop until '('.
operator -> Pop higher/equal precedence operator, then push
Pop remaining operator.

POSTFIX TO INFIX:
Scan expression left to right
operand -> Push
operator -> Pop two operands
Combine as (operand1 operator operand2).
Push result
Final stack element = infix expression.

* Time Complexity

operation	Time Complexity
Push	O(1)
Pop	O(1)
Infix -> Postfix	O(n)
Postfix -> Infix	O(n)

* FAQ's

1. What is advantage of prefix and postfix over infix expression.

→ Prefix and postfix expressions do not require parentheses and do not need operator precedence rules during evaluation. They are faster and easier for computers to evaluate using a stack.

2. Why is stack used in expression conversion?

→ The stack follows LIFO (Last In, First Out). It is useful for temporarily storing operators and operands and handling their correct order during infix, prefix and postfix conversions.

3. Write stack full and stack empty conditions for stack.

→ Stack full: when TOP = MAX - 1 no more element can be inserted.
Stack Empty: when Top = -1 there are no elements in the stack.

* Conclusion:
Thus, implemented stack operations assignment using array concept. This system is able to apply different expression.

PRACTICAL 7

EXPRESSION CONVERSION USING STACK

CSF20030 – Data Structures Laboratory

Experiment 07: Infix to Postfix Conversion

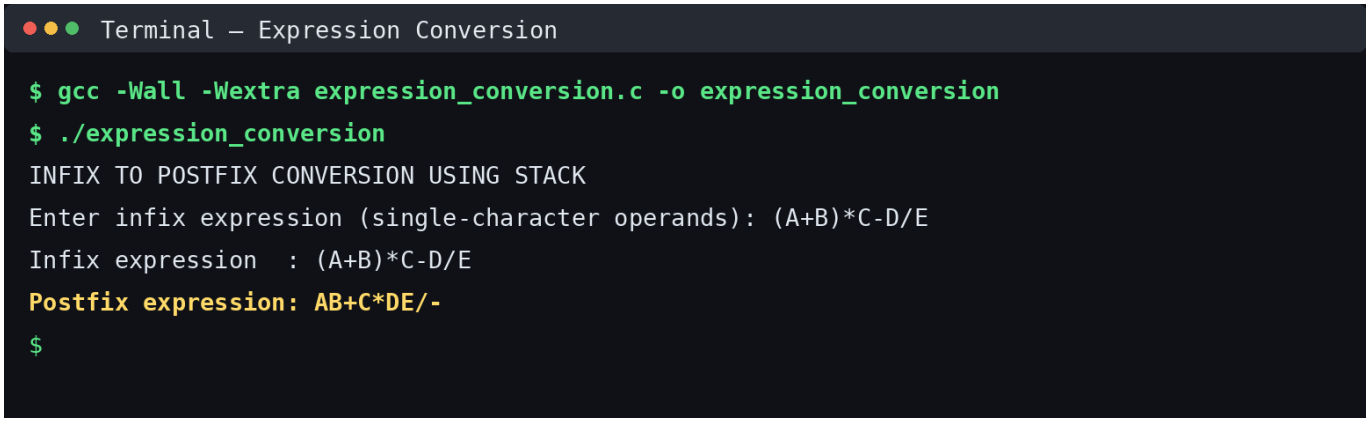
Aim: Implement a stack and use it to convert an infix expression to postfix notation.

C Program

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#define MAX 200
char stack[MAX];
int top = -1;
void push(char ch) {
    if (top == MAX - 1) {
        printf("Error: Stack overflow.\n");
        exit(EXIT_FAILURE);
    }
    stack[++top] = ch;
}
char pop(void) {
    if (top == -1) {
        printf("Error: Stack underflow.\n");
        exit(EXIT_FAILURE);
    }
    return stack[top--];
}
char peek(void) {
    if (top == -1) return '\0';
    return stack[top];
}
int precedence(char op) {
    switch (op) {
        case '^': return 3;
        case '*':
        case '/':
        case '%': return 2;
        case '+':
        case '-': return 1;
        default: return 0;
    }
}
/* Convert an infix expression to postfix.
   Operands should be single letters/digits; spaces are ignored. */
void infixToPostfix(const char infix[], char postfix[]) {
    int j = 0;
    top = -1;
    for (int i = 0; infix[i] != '\0'; i++) {
        char ch = infix[i];
        if (isspace((unsigned char)ch)) continue;
        if (isalnum((unsigned char)ch)) {
            postfix[j++] = ch;
        } else if (ch == '(') {
            push(ch);
        } else if (ch == ')') {
            while (top != -1 && peek() != '(')
                postfix[j++] = pop();
            if (top == -1) {
                printf("Error: Mismatched parentheses.\n");
                exit(EXIT_FAILURE);
            }
            pop(); /* discard '(' */
        } else if (strchr("+-*/%^", ch) != NULL) {
            while (top != -1 && peek() != '(' &&
                   (precedence(peek()) > precedence(ch) ||
                   (precedence(peek()) == precedence(ch) && ch != '^'))) {
                postfix[j++] = pop();
            }
            push(ch);
        } else {
            printf("Error: Invalid character '%c'.\n", ch);
            exit(EXIT_FAILURE);
        }
    }
    while (top != -1) {
        char op = pop();
        if (op == '(') {
            printf("Error: Mismatched parentheses.\n");
            exit(EXIT_FAILURE);
        }
        postfix[j++] = op;
```

```
    }
    postfix[j] = '\0';
}
int main(void) {
    char infix[MAX], postfix[MAX];
    printf("INFIX TO POSTFIX CONVERSION USING STACK\n");
    printf("Enter infix expression (single-character operands): ");
    if (fgets(infix, sizeof(infix), stdin) == NULL) {
        printf("Input error.\n");
        return 1;
    }
    infixToPostfix(infix, postfix);
    printf("Infix expression  : %s", infix);
    printf("Postfix expression: %s\n", postfix);
    return 0;
}
```

Sample Input and Output Screenshot



Input: (A+B)*C-D/E
Output: AB+C*DE/-
The screenshot illustrates the compiled program converting the infix expression into postfix form.